

ABES Engineering College, Ghaziabad
Affiliated to Dr. APJ AKTU Lucknow

Department of CSE



Lab Manual

Session 2026-27 (Odd Semester)

Name : Yuvraj Mishra

Subject Name : Design and Analysis of Algorithms Lab

Subject Code : BCS-553

Semester : BTech (CSE) V SEMESTER

EXPERIMENT #1

Problem Statement: Write a program to search an element in an array using recursive binary and linear search method.

Linear Search

Objective: Linear search method is searching an element in an array linearly. It will take $O(n)$ time.

Input: An array $A[]$, element to be searched

Algorithm:

```
low = 0, high = N-1;
LinearSearch(A[0..N-1], value, low, high)
{ if (low <= high) {
    if (A[low] ==
        value) return
        low;
    else
        return LinearSearch(A[0..N-1], value, low+1, high);
}
```

Binary Search

Objective: When the array is sorted, binary search method is the best method to search an element in $O(\log n)$ time using Divide & conquer method.

Inputs: A sorted array $A[]$, element to be searched

Algorithm:

```
low = 0
high = N - 1
BinarySearch(A[0..N-1], value, low, high) {
    if (low <= high) {
        mid = (low + high) / 2
        if (A[mid] == value)
            return mid;
        else if (A[mid] > value)
            BinarySearch(A[0..N-1], value, low, mid-1)
        else if (A[mid] < value)
            BinarySearch(A[0..N-1], value, mid+1, high)
    }
    return not_found;
```

CODE-:

```
#include<iostream>
using namespace std;

int recursiveLinearSearch(int array[],int key,int size){
size=size-1;
if(size <0)
{
return -1;
}
else if(array[size]==key)
{
return size;
}
else
{
return recursiveLinearSearch(array,key,size);
}
}

int binarySearch(int arr[], int l, int r, int x)
{ if (r >= l) { int mid = l
+ (r - l) / 2;
if (arr[mid] == x)
return mid;

if (arr[mid] > x)
return binarySearch(arr, l, mid - 1, x);

return binarySearch(arr, mid + 1, r, x);
}
return -1;
}

int main()
{
cout<<"Enter The Size Of Array:-"; int
size; cin>>size; int array[size], key,i;
```

```

cout<<"Enter The Elements Of Array:-
\n"; for(int j=0;j<size;j++)
{
cin>>array[j];
}
cout<<"Enter Key To Search in Array->"; cin>>key; int ch; cout<<"Enter
1 for Linear Search\nEnter 2 for Binary Search\n\n"; cin>>ch;
switch(ch)
{
case 1:
    int result;
    result=recursiveLinearSearch(array,key,size
--); if(result!=-1){ cout<<"Element Found in
Array at "<<result+1; break;
}
else{ cout<<"Element NOT Found
in Array "; break;
}
case 2:
    result = binarySearch(array, 0, size - 1, key);
    (result == -1) ? cout << "Element is not present in array"
        : cout << "Element is present at " << result+1;
}

    return 0;
}

```

OUTPUT:-

```

Enter The Size Of Array:-5
Enter The Elements Of Array:-
1
2
3
4
5

Enter Key To Search in Array:-4
Enter 1 for Linear Search
Enter 2 for Binary Search

1
Element found in Array at 4
Process returned 0 (0x0)   execution time : 32.778 s
Press any key to continue.

```

Yuvraj Mishra 2400320101312

EXPERIMENT #2

Problem Statement: Write a program to sort an array using Merge Sort.

Objectives: MergeSort is good for sorting slow-access data. It is excellent for sorting data that normally accessed sequentially eg. linked list, tape drive, hard disk and receiving online data one at a time.

Algorithm:

```
MERGE-SORT( $A, p, r$ )
1  if  $p < r$ 
2       $q = \lfloor (p + r)/2 \rfloor$ 
3      MERGE-SORT( $A, p, q$ )
4      MERGE-SORT( $A, q + 1, r$ )
5      MERGE( $A, p, q, r$ )

MERGE( $A, p, q, r$ )
1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1 \dots n_1 + 1]$  and  $R[1 \dots n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
```

CODE-:

```
#include<iostream> using namespace
std; void merge (int arr[], int l, int m,
int r)
{ int i, j, k; int n1 = m -
    l + 1; int n2 = r
    - m; int L[n1],
    R[n2];

    for (i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    i =
    0; j
    = 0;
    k =
    l;
    while (i < n1 && j < n2)
    { if (L[i] <= R[j])
        { arr[k] = L[i]; i++;
        }
        else
        { arr[k] = R[j]; j++;
        }
        k++;
    }
    while (i < n1)
    { arr[k] = L[i]; i++;
        k++;
    }
    while (j < n2)
    {
        arr[k] =
        R[j]; j++;
        k++;
    }
}
```

```

}
void mergeSort(int arr[], int l, int r)
{ if (l < r)
    { int m = l + (r - l) / 2;
      mergeSort(arr, l, m);
      mergeSort(arr, m + 1, r); merge(arr, l, m, r);
    }
}

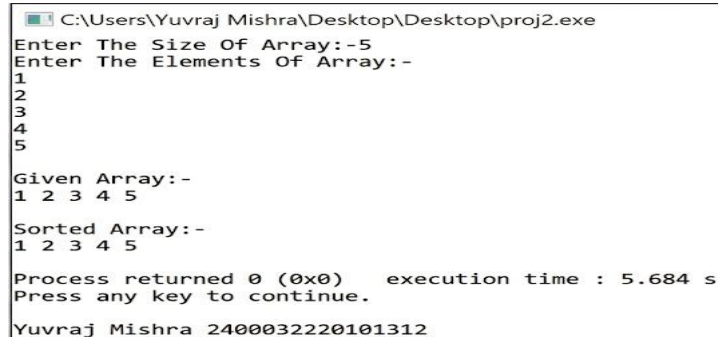
int main()
{
    cout<<"Enter The Size Of Array:-"; int
size; cin>>size; int arr[size], key,i;
    cout<<"Enter The Elements Of Array:-
\n"; for(int j=0;j<size;j++)
    {
cin>>arr[j];
    }

    cout<<"Given Array--\n";
    for (i = 0; i < size; i++)
        cout<<arr[i]<<" ";
    mergeSort(arr, 0,size - 1);

    cout<<"\n\nSorted    Array--
\n"; for (i = 0; i < size; i++)
        cout<<arr[i]<<" "; return0;}

```

OUTPUT:-



```

C:\Users\Yuvraj Mishra\Desktop\Desktop\proj2.exe
Enter The Size Of Array:-5
Enter The Elements Of Array:-
1
2
3
4
5

Given Array:-
1 2 3 4 5

Sorted Array:-
1 2 3 4 5

Process returned 0 (0x0)   execution time : 5.684 s
Press any key to continue.
Yuvraj Mishra 2400032220101312

```

EXPERIMENT #3

Problem Statement: Write a program to sort an array using QuickSort.

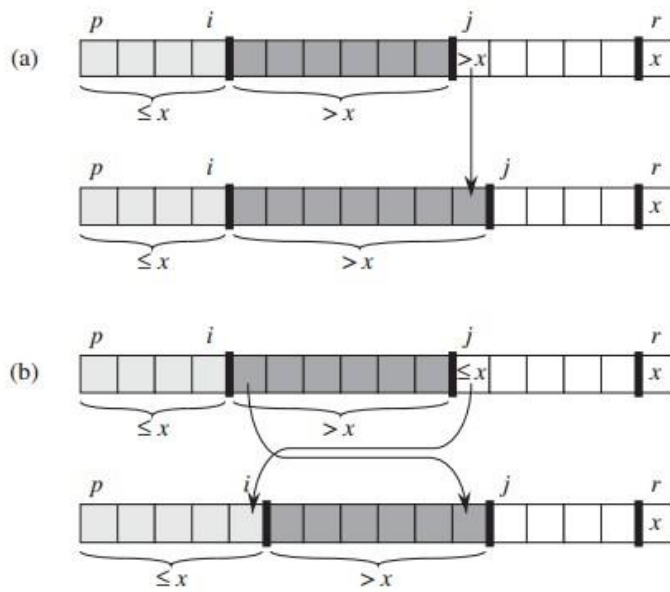
Objectives: Quicksort is an inplace sorting algorithm, It is brilliant as it's inner loop can be efficiently implemented on most computer architectures.

Algorithm:

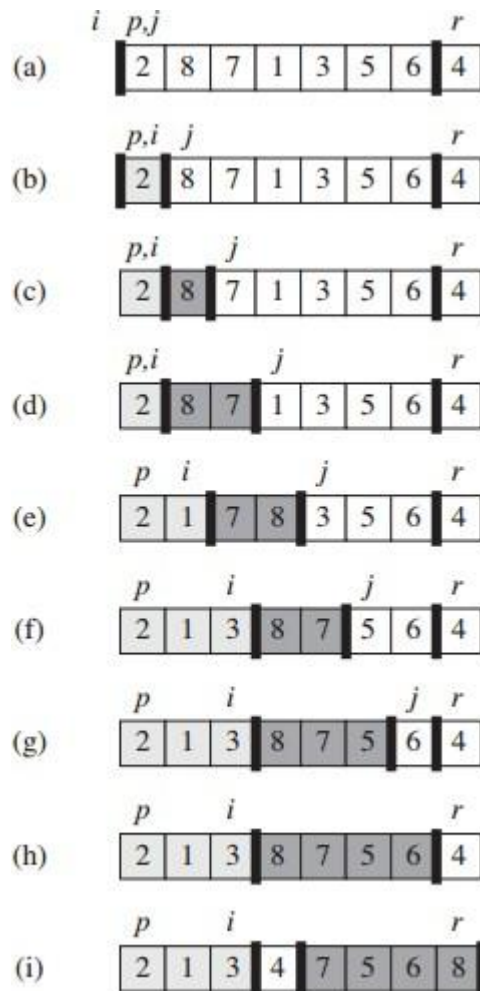
```
QUICKSORT( $A, p, r$ )
1  if  $p < r$ 
2       $q = \text{PARTITION}(A, p, r)$ 
3      QUICKSORT( $A, p, q - 1$ )
4      QUICKSORT( $A, q + 1, r$ )
```

```
PARTITION( $A, p, r$ )
1   $x = A[r]$ 
2   $i = p - 1$ 
3  for  $j = p$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i = i + 1$ 
6          exchange  $A[i]$  with  $A[j]$ 
7  exchange  $A[i + 1]$  with  $A[r]$ 
8  return  $i + 1$ 
```

Two cases of one iteration of partition:



E



CODE-:

```
#include<bits/stdc++.h>          using
namespace std;
int partition(int a[],int start,int end)
{
    int pivot=a[end]; int
    index=start; int temp;
    for(int i=start;i<end;i++)
    {
        if(a[i]<=pivot)
        {
            temp=a[i];          a[i]=a[index];
            a[index]=temp; index++;
        }
    }

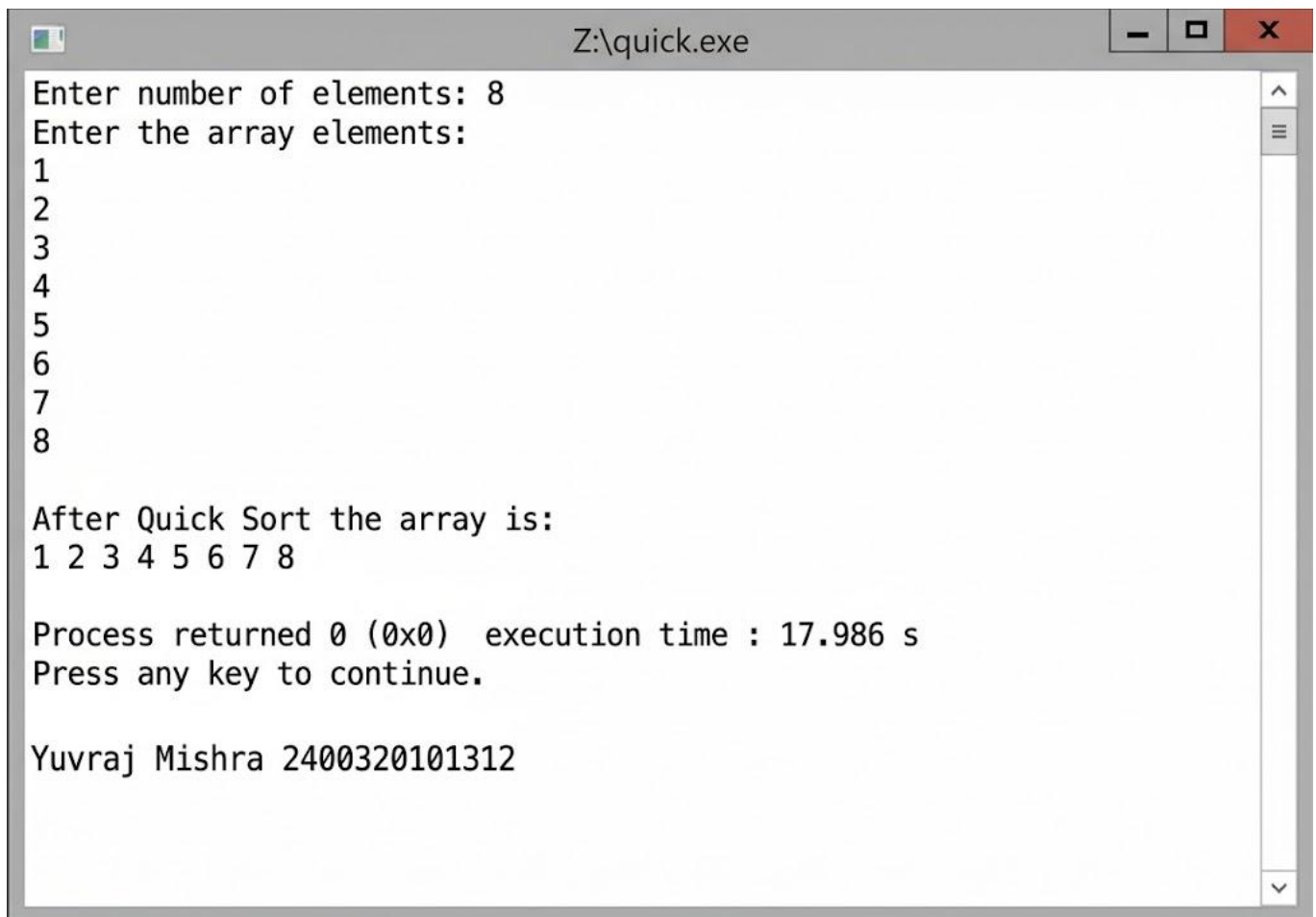
    temp=a[end];
    a[end]=a[index];
    a[index]=temp;

    return index;
}
void Quicksort(int a[],int start,int end)
{
    if(start<end)
    {
        int          index=partition(a,start,end);
        Quicksort(a,start,index-1);
        Quicksort(a,index+1,end);
    }
}
int main()
{ int n; cout<<"Enter number of elements: ";
  cin>>n; int a[n];
  cout<<"Enter the array elements:\n"; for(int
                                  i=0;i<n;i++)

  {
      cin>>a[i];
  }
}
```

```
Quicksort(a,0,n-1);  
cout<<"After Quick Sort the array is:\n"; for(int  
i=0;i<n;i++)  
{  
    cout<<a[i]<<" ";  
}  
return 0;  
}
```

OUTPUT:-



```
Z:\quick.exe  
Enter number of elements: 8  
Enter the array elements:  
1  
2  
3  
4  
5  
6  
7  
8  
  
After Quick Sort the array is:  
1 2 3 4 5 6 7 8  
  
Process returned 0 (0x0) execution time : 17.986 s  
Press any key to continue.  
  
Yuvraj Mishra 2400320101312
```

EXPERIMENT #4

Problem Statement: Write a Program to sort an array using Insertion sort.

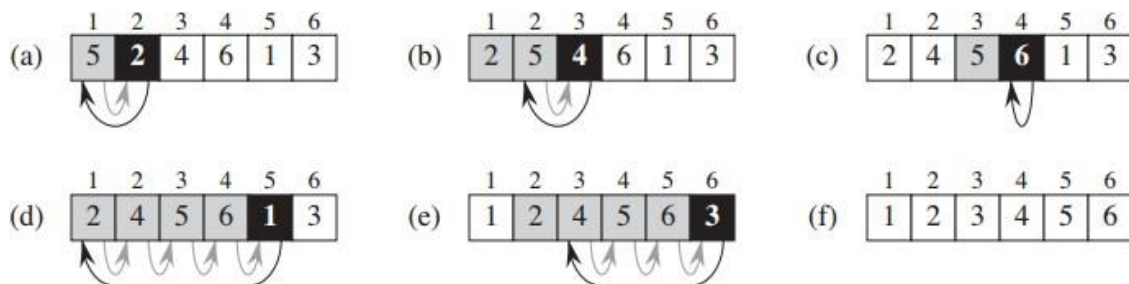
Objectives: The insertion sort is useful when the array is nearly sorted or the array size is small. When humans manually sort something (for example, a deck of playing cards), most use a method that is similar to insertion sort.

Inputs: An array A[] **Algorithm:**

INSERTION-SORT(A)

```
1  for  $j = 2$  to  $A.length$ 
2       $key = A[j]$ 
3      // Insert  $A[j]$  into the sorted sequence  $A[1..j-1]$ .
4       $i = j - 1$ 
5      while  $i > 0$  and  $A[i] > key$ 
6           $A[i + 1] = A[i]$ 
7           $i = i - 1$ 
8       $A[i + 1] = key$ 
```

Example:



CODE:-

```
#include<iostream>

#include<conio.h>    using
namespace std; main() { int
Key,n; cout<<"Enter size of
Array:"; cin>>n; int array[n];
cout<<"Enter      "<<n<<"
numbers:
"<<endl; for(int i=0;i<n;i++)
```

```

{ cin>>array[i]; }

cout<<"\nWithout

Sort:"<<endl; for(int j=0; j<n;

j++)

{ cout<<array[j]<<" ";

} for(int j=1 ; j<n ;

j++)

{ Key =

    array[j]; int i

    = j-1;

    while(i>=0&&array[i]>Key)

    {

        array[i+1]=array[i]; i=i-1;

    } array[i+1]=Key; }

cout<<"\nSorted Array

is:

"<<endl; for(int i=0; i<n; i++)

{ cout<<array[i]<<" ";

}}

```

OUTPUT:-

```

Enter size of Array:-5
Enter 5 numbers:
8
0
1
7
2

Without Sort:-
8 0 1 7 2

Sorted Array is:
0 1 2 7 8

Process returned 0 (0x0)    execution time : 8.325 s
Press any key to continue.

Yuvraj Mishra 2400320101312

```

EPERIMENT #5

Problem Statement: Write a program to sort an array using HeapSort.

Objectives: Heapsort is an efficient sorting algorithm with $O(n \log n)$ worst case time complexity. It is an in-place sorting algorithm.

Algorithm:

HEAPSORT(A)

```
1  BUILD-MAX-HEAP( $A$ )
2  for  $i = A.length$  downto 2
3      exchange  $A[1]$  with  $A[i]$ 
4       $A.heap-size = A.heap-size - 1$ 
5      MAX-HEAPIFY( $A, 1$ )
```

BUILD-MAX-HEAP(A)

```
1   $A.heap-size = A.length$ 
2  for  $i = \lfloor A.length/2 \rfloor$  downto 1
3      MAX-HEAPIFY( $A, i$ )
```

MAX-HEAPIFY(A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.heap-size$  and  $A[l] > A[i]$ 
4       $largest = l$ 
5  else  $largest = i$ 
6  if  $r \leq A.heap-size$  and  $A[r] > A[largest]$ 
7       $largest = r$ 
8  if  $largest \neq i$ 
9      exchange  $A[i]$  with  $A[largest]$ 
10     MAX-HEAPIFY( $A, largest$ )
```

PARENT(i)

```
1  return  $\lfloor i/2 \rfloor$ 
```

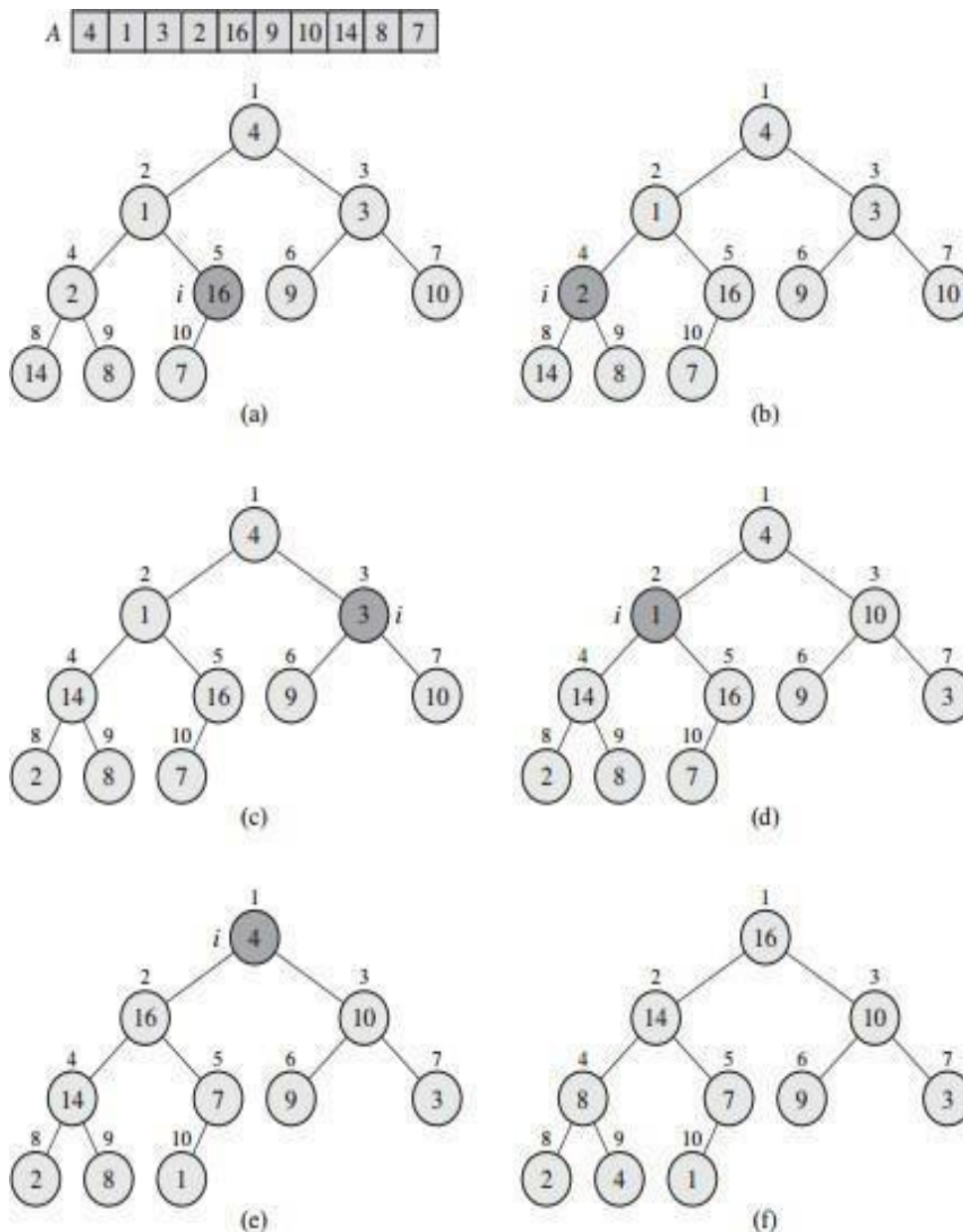
LEFT(i)

```
1  return  $2i$ 
```

RIGHT(i)

```
1  return  $2i + 1$ 
```

Example showing operations of BUILD MAX HEAP:



CODE:-

```
#include <iostream> using
namespace std;
void heapify(int arr[], int n, int
i) { int largest = i;

    int left = 2 * i + 1; int
    right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest]) largest
        = left;

    if (right < n && arr[right] > arr[largest])
```

```

        largest = right; if
(largest != i) {

    swap(arr[i], arr[largest]);
    heapify(arr, n, largest);
}

}

void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--) heapify(arr,
        n, i);
    for (int i = n - 1; i >= 0; i--)
    { swap(arr[0], arr[i]);
        heapify(arr, i, 0);

    }
}

int main()
{ int

    n;

    cout<<"Enter size of Array-:";
    cin>>n; int array[n];

    cout<<"Enter "<<n<<" numbers: "<<endl;
    for(int i=0;i<n;i++)
    {
        cin>>array[i];
    }

    cout<<"\nWithout Sort-:"<<endl;
    for(int j=0; j<n; j++)
    {
        cout<<array[j]<<" ";
    }
    heapSort(array, n); cout
    <<"\nSorted array is \n";
    for(int j=0; j<n; j++)
    {

        cout<<array[j]<<" ";

    }
}

```

OUTPUT:

Enter size of Array:-5

Enter 5 numbers:

7

0

1

8

3

Without Sort:-

7 0 1 8 3

Sorted array is

0 1 3 7 8

Process returned 0 (0x0) execution time : 6.311 s

Press any key to continue.

Yuvraj Mishra 2400320101312

|

